

MultiVI at Atlas Scale: Zarr Streaming in scvi-tools-2

Bounded-memory training + dense ADT lazy loading

Outline

Two major changes in scvi-tools-2

1. Zarr-backed streaming dataloader
2. Dense ADT `zarr.Array` with mixed RNA CSR + ADT dense layouts

Agenda: Problem → Architecture → ADT memory → Benchmarks → Usage

The Problem

Atlas-scale MuData (10–20M cells) cannot fit all modality matrices in RAM

RNA: sparse counts work well as zarr-backed CSR

ADT / protein: effectively dense; CSR is a poor fit at this scale

MultiVI training still requires dense minibatches on GPU

Default DataSplitter does not scale the same way for multi-modality zarr MuData

Change 1: Zarr Streaming Architecture

New in scvi-tools-2 (not upstream scvi-tools):

ZarrDataset — IterableDataset with sorted block reads + shuffle buffer

ZarrMultiViDataModule — Lightning datamodule for MultiVi

MULTIVI.train() auto-selects zarr datamodule for fully zarr-backed MuData

Pipeline: zarr store → block reads → shuffle buffer → minibatch → train

How Streaming Achieves Efficiency

Sorted within-block reads avoid random zarr seeks

Shuffle buffer holds $\text{block_size} \times \text{shuffle_buffer_blocks}$ rows, not full data

Workers reopen zarr via picklable ZarrMatrixSource descriptors

DDP: disjoint per-rank slices with equal batch counts

Defaults: $\text{block_size}=4096$, $\text{shuffle_buffer_blocks}=16$

Change 2: Dense ADT Memory (Table)

Scale	Full dense ADT in RAM	Zarr block (4096 rows)
10M cells × 300 ADTs	~12 GB (float32)	~5 MB per block
20M cells × 300 ADTs	~24 GB (float32)	~5 MB per block

Change 2: Dense ADT Memory (Notes)

At 10–20M cells, loading full ADT into RAM blows memory

Backed dense `zarr.Array` lazy-loads only rows for the current block/batch

ADT is dense-ish: CSR adds index overhead without benefit

Use `matrix_layout='auto'` for RNA CSR + ADT dense on `.X`

Caveat: `from_backed_mudata()` defaults to `'csr'`; `MULTIVI.train` auto-path uses `'auto'`

What Changed in Code

New: `_zarr_dataset.py`, `_zarr_datamodule.py`

`MULTIVI.train()`: auto zarr datamodule + `reload_dataloaders_every_n_epochs=1`

zarr.Array support in count validation helpers

Docs: `docs/user_guide/use_case/zarr_multivi_streaming.md`

Commits: `fcd3b31d8`, `1c2c96273`, `517c16aad`

Benchmark: CSR vs Zarr (Results)

Metric	CSR DataSplitter	Zarr streaming	Speedup
Loader batches/s (steady)	13.2	104.2	7.9×
Training batches/s	4.6	7.2	1.55×
Wall time (100 batches)	21.6 s	13.9 s	1.55×
First-batch latency	0.79 s	1.62 s	CSR faster

Benchmark: CSR vs Zarr (Setup)

Source: `scripts/benchmark_multivi_csr_vs_zarr_wsl_results.json`

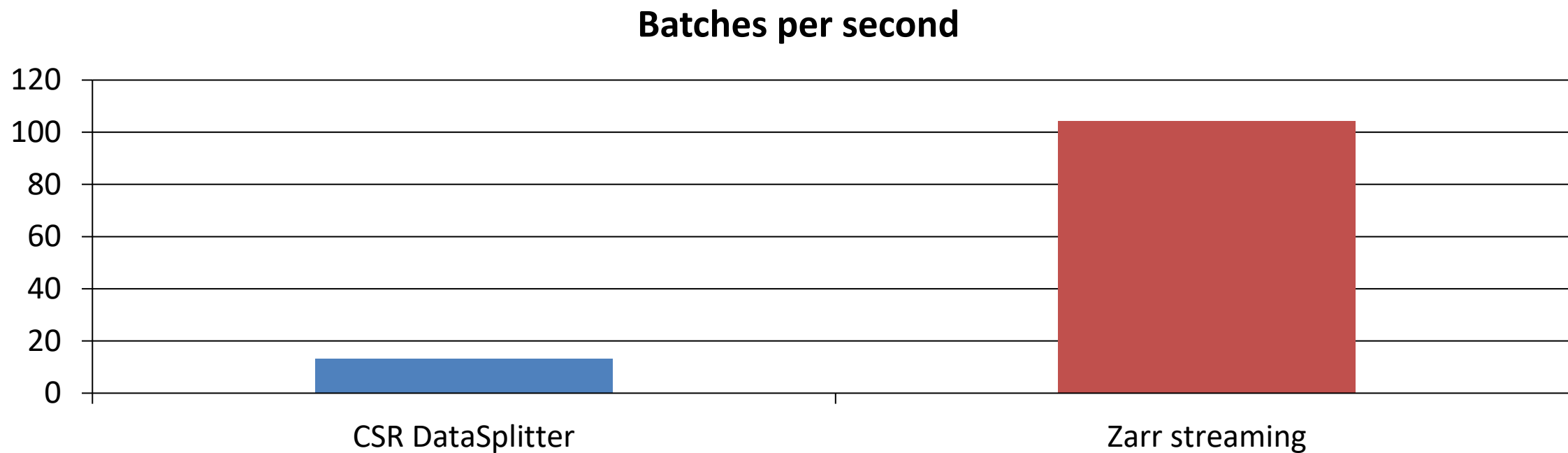
19,200 cells; 2,000 genes + 2,000 ATAC regions

GPU: RTX 2000 Ada; `pin_memory=True`; `prefetch_to_gpu=True`

Zarr wins sustained throughput and total training time

First batch slower due to zarr reopen / worker startup

Benchmark Chart: Loader Throughput



~7.9× higher steady-state loader throughput with ZarrMultiVIDataModule

End-to-end training speedup ~1.55× because GPU compute dominates

Benchmark: Loader Prefetch / Tuning

Config	Wall (150 batches)	Loader wait	Batches/s
Baseline (num_workers=0)	21.9 s	1.30 s	6.85
Workers + pin_memory	18.8 s	0.16 s	7.96
Large batch (512)	5.9 s	0.19 s	25.3

Benchmark: Prefetch Notes

Source: `scripts/benchmark_multivi_idle_gap_wsl_results.json`

Same ~19k-cell synthetic trimodal MuData on GPU

`num_workers` + `pin_memory` sharply reduces GPU idle waiting

Batch size has a large effect on throughput

Positioning vs TileDB-SOMA / CZI

Shared goal: bounded memory + high-throughput streaming

Zarr: MuData zarr stores, MultiVI, mixed CSR + dense modalities

TileDB-SOMA: Census-scale SCVI/SCANVI via SOMA queries

Choose Zarr when atlases are already in mudata.write_zarr format

How to Use

Write MuData to zarr and reopen with backed modality .X

MULTIVI.setup_mudata(...) with correct modality mapping

Mixed RNA + ADT: from_backed_mudata(..., matrix_layout='auto', store_dir=...)

Or model.train(...) on fully zarr-backed MuData (auto datamodule)

Tune: block_size, shuffle_buffer_blocks, num_workers, pin_memory

Limitations

EXPERIMENTAL — MultiVI only

No categorical / continuous covariates yet

CSR modalities densified per batch before GPU (expected)

Windows often needs num_workers=0

Summary & Next Steps

Pillar 1: Zarr streaming for bounded-memory MultiVI training

Pillar 2: Dense ADT zarr.Array for 10–20M-cell lazy loading

Headline benchmarks: 7.9× loader throughput; 1.55× training speedup

Next: 10M+ cell ADT benchmarks, API docs, tutorial notebook